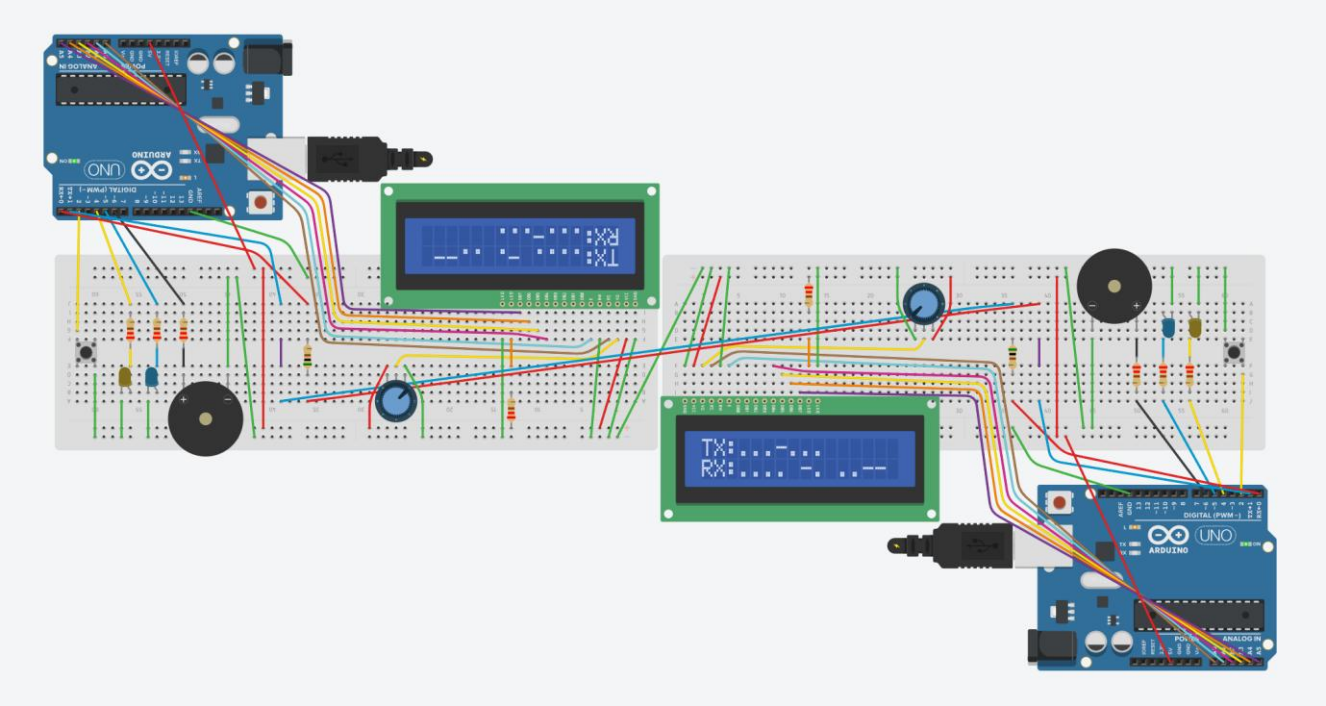
Morse Telegraph

Intermediate Lab with LCD screens

Build a pair of identical Morse Code messaging terminals in a group. Each has a pushbutton, two LEDs, a piezo buzzer, a 10kΩ potentiometer, and a 16x2 LCD display. One button lights your Transmit (TX) LED and the other terminal's Receive (RX) LED; both buzzers sound while either key is held. The top row of each screen records outgoing Morse; the bottom row records the incoming Morse. You will wire digital inputs, control outputs, and send information between two Arduino Uno R3 microcontrollers over a serial link.



Intermediate circuit reference from TinkerCAD. The picture places both terminals on separate breadboards. One LCD and Uno are rotated; identify pins by their printed labels. The local LCD wiring matches the table in this manual.

**This manual and its included sketch are for the physical Uno build in Arduino IDE. The confirmed physical serial resistors are 10 kΩ; the working TinkerCAD serial links use 500 Ω. Additionally, the TinkerCAD version uses D0/D1 for each serial connection, while the physical wiring will remain D8/D9 as listed in this lab manual.*

The help section at the end of the manual explains the concepts used along the way and provides a guide to using Morse Code.

Parts and pin connections

Start with the parts below. Each group needs two complete terminals; the pair column gives the total for your group.

Part	One terminal	Pair
Arduino Uno R3	1	2
Breadboard	1	2
Pushbutton	1	2
LEDs in two colors for TX and RX	2	4
220 Ω resistors for the indicator LEDs	2	4
10 kΩ resistor for the outgoing serial line	1	2
Small passive piezo buzzer and a 220 Ω series resistor	1 of each	2 of each
USB data cable and powered USB port	1	2
16x2 parallel LCD display with fitted header	1	2
3-pin 10 kΩ potentiometer	1	2
220 Ω resistor for the LCD backlight	1	2
Jumper wires		~50-60

Also needed: at least one computer with Arduino IDE, the “AltSoftSerial” Library in Arduino IDE by Paul Stoffregen for the serial connections, and the “LiquidCrystal” Library in Arduino IDE by Arduino, Adafruit.

Wire each terminal the same way

Uno pin	Connection	Job
5V	Local power rail	Powers LCD screen
GND	Local ground rail	Return path for the key, LEDs and piezo
D2	One switched button contact; the other contact to GND	Input; LOW when pressed
D4	220 Ω → TX LED anode; LED cathode to GND	Output; your own button
D5	220 Ω → RX LED anode; LED cathode to GND	Output; the other button
D6	220 Ω → passive piezo +; piezo − to GND	Output; 800 Hz sound
D8	Other Uno D9 through one 10 kΩ resistor	RX input from the other terminal
D9	One 10 kΩ resistor → other Uno D8	TX output to the other terminal

The serial pins in this sketch are fixed at D8 and D9. Each Arduino board gets its own USB power. Share ground and the two serial signal paths; keep their positive supply rails separate. A0–A5 will now be occupied by the LCD screen as listed in the LCD pin map section.

Build each terminal

**Leave the USB power unplugged while wiring!*

**Label each board/terminal A and B respectively for easier troubleshooting.*

**'L' is the small LED already on the Uno, separate from your two breadboard LEDs.*

1 Set up the breadboard rails

Connect Uno GND to a ground rail and Uno 5V to a separate positive rail. Groups of five holes are internally connected; the center gap separates the two halves. Long power rails may have a break; check continuity with power removed and bridge a break only if you need both sections. Never join the two Unos' positive rails.

2 Wire the pushbutton to D2

Place a four-leg button across the breadboard center gap. Two legs on one internal contact are already joined; pressing connects the two contacts. Connect D2 to one contact and GND to the other. If unsure, use a continuity meter with power removed: the chosen contacts should connect only when pressed.

3 Add the two indicator LEDs

Assign one LED TX and the other RX. Connect D4 through its own 220 Ω resistor to the TX LED anode (+), then its cathode (−) to GND. Repeat from D5 through a separate 220 Ω resistor for RX. On a typical LED the longer leg is the anode; and the shorter leg is the cathode.

4 Add the passive piezo buzzer

Connect D6 through a 220 Ω resistor to the supplied passive piezo + terminal; connect its − terminal to GND. Follow the markings on the part. This circuit uses a small passive piezo, not a bare loudspeaker. Either your key or a received key press will make it sound.

5 Wire the LCD and contrast control

Use the complete LCD table on the next page. Connect the screen and potentiometer to this terminal's rails, then add the six signal wires to A0–A5. Include the separate 220 Ω backlight resistor. Repeat the same construction for the other terminal.

LCD pin map

Unplug both boards before wiring. Read the labels on the display itself; viewing the back of the module reverses the apparent order. Wire each LCD to its own Uno and local rails.

LCD pin	Label	Connect to	Purpose
1	VSS	Local GND	Logic ground
2	VDD	Local 5V	Logic supply
3	V0 or VO	10 kΩ potentiometer wiper	Contrast
4	RS	Uno A0	Choose instruction or character data
5	RW	Local GND	Write mode
6	E	Uno A1	Latch the transfer
7–10	D0–D3	Leave unconnected	Unused in four-bit mode
11	D4	Uno A2	Display data
12	D5	Uno A3	Display data
13	D6	Uno A4	Display data
14	D7	Uno A5	Display data
15	A or LED+	Local 5V through 220 Ω	Backlight anode
16	K or LED−	Local GND	Backlight cathode

Connect the contrast potentiometer

Potentiometer terminal	Connection
One outer terminal	Local 5V
Other outer terminal	Local GND
Center wiper	LCD V0 at pin 3

The knob adjusts the LCD contrast. Reversing the two outer connections only reverses the direction of adjustment. Adjust the knob until the desired contrast is met. Keep the backlight's 220 Ω resistor in place; backlight brightness and character contrast are separate things.

LCD D4 is a display label, not Uno D4. This lab connects LCD D4 to Uno A2. The C++ constructor below lists RS, E, then LCD D4, D5, D6 and D7 in that order.

```
LiquidCrystal lcd(A0, A1, A2, A3, A4, A5);
```

Use one resistor in each physical serial path

From	Through	To
A D9 TX	One 10 kΩ resistor	B D8 RX
B D9 TX	One 10 kΩ resistor	A D8 RX
A ground rail	One jumper	B ground rail

***Leave the two serial signal wires disconnected until both terminals pass their individual tests on the next page. Connect the pair during Step 4 of "Upload to each board and test."*

Upload to each board and test

1 Prepare the software

Connect one Uno with a USB data cable. In Arduino IDE, select Arduino Uno and its port under Tools. Install AltSoftSerial by Paul Stoffregen and LiquidCrystal by Arduino in Library Manager if missing. Open the intermediate .ino file or paste the complete code that follows in this manual. Click Verify, then Upload.

2 Check terminal A on its own

With the serial link wires unplugged, power A with its key released. Slowly adjust contrast until 'TX:' and 'RX:WAIT LINK' are readable. Press the pushbutton: the TX LED and piezo should turn on during the press and stop on release. A roughly 0.2 s press should add a dot; a 0.6 s press a dash.

3 Upload and check terminal B

Select B's USB port and upload the same intermediate sketch. Repeat the local checks. Identify the correct port by unplugging and reconnecting one board if necessary. Fix local faults before joining the pair: check the button contacts if always pressed, LED polarity if dark, and contrast and LCD wiring if text is missing.

4 Connect and test the pair

Unplug both USB cables. Add A D9 → 10 kΩ → B D8, B D9 → 10 kΩ → A D8, and the shared GND connection from the serial table. Power both with keys released. WAIT LINK should disappear and both built-in L LEDs should light. Press A only: A TX and B RX should light and both piezos sound. Release, compare A TX history with B RX, then repeat from B.

If either RX row stays at WAIT LINK, check peer power, common GND, crossed D9/D8 paths and the resistor connections.

Practice and check the pair

Read the screen

Each line has a three-character label and room for 13 history characters. TX is your own key; RX is your partner's key. A symbol appears when the key is released. The LEDs and sound respond during the hold.

RX display	Meaning	What to do
WAIT LINK	No valid incoming D/U has arrived since startup.	Check peer power, crossed D9/D8 wires, resistors and common GND.
RELEASE KEY	The receiver first encountered an already-held key.	Have the sender release, then start a fresh press.
LINK LOST	No valid update for 500 ms; RX history was cleared.	Restore the connection, release the keys and restart the pattern.

The program measures duration in milliseconds, so 200 ms is 0.2 seconds. Spaces and slashes appear when the next valid button press is released. After completing the tests, try changing 'DOT_UNIT_MS' on both boards. This changes the dot/dash cutoff and the letter and word gap thresholds together.

Measured time	What the sketch does	Try this
Hold under 50 ms	No symbol	Avoid tiny taps
Hold 50–399 ms	Adds a dot on release	Hold about 0.2 s
Hold 400 ms or longer	Adds a dash on release	Hold about 0.6 s
Gap under 600 ms	Keeps symbols together	About 0.2 s inside a letter
Gap 600–1399 ms	Inserts a letter space	About 0.8 s between letters
Gap 1400 ms or longer	Inserts space slash space	About 1.6 s between words

Look inside the program (optional)

Open Serial Monitor in Arduino IDE at 115200 baud. Presses produce LOCAL DOWN and LOCAL UP; incoming changes produce REMOTE DOWN and REMOTE UP. Send ? to see state and byte counts. The invalid count should normally remain zero. Each L LED reports incoming traffic only, so use the keys to check both directions.

Link interruption (optional)

Keep both boards powered and the common ground connected. With B's key released and A held, remove only the wire entering B D8. Within about half a second B RX and B's piezo should turn off. Reconnect that wire: B should follow A again. The LED and sound resume, but the interrupted press should not add a received symbol. Release A, then make a fresh press to resume the RX history. Keep the loose end away from other pins. Unplug both boards for ordinary rewiring.

Before moving on: demonstrate both directions and one overlapping hold. Write down one wiring mistake you found or one behavior you can now explain.

Observation(s):

_____

_____

_____

Complete sketch

Open Telegraph_Terminal_Intermediate.ino or paste this complete listing into Arduino IDE as plain text. Copy through the final closing brace. Upload the same sketch to both boards.

```
/*
  IEEE Student Branch - Intermediate Morse Telegraph
  Arduino Uno R3 / ATmega328P. Load the same sketch on both boards.

    D2 -> normally open button -> GND (LOW means pressed)
    D4 -> 220 ohms -> TX LED anode; cathode -> GND
    D5 -> 220 ohms -> RX LED anode; cathode -> GND
    D6 -> 220 ohms -> small passive piezo +; piezo - -> GND
    A D9 -> 10k ohms -> B D8; B D9 -> 10k ohms -> A D8
    Join the grounds. Power both Unos by USB; keep their 5V separate.

  AltSoftSerial: fixed D8 RX / D9 TX, 9600 baud. 'D' = down; 'U' = up.
  USB: 115200 baud; send ? for status. L = recent valid incoming data.
  Serial resistors: physical 10k ohms; Tinkercad 500 ohms.

  LCD wiring: RS=A0, E=A1, LCD D4=A2, D5=A3, D6=A4, D7=A5.
  LCD VSS/RW/K -> GND; VDD -> 5V; A -> 220 ohms -> 5V.
  V0 -> 10k potentiometer wiper; its ends go to 5V and GND.
  LCD D0-D3 stay unused. Each LCD uses its own Uno's 5V.
  Each row keeps the latest 13 symbols and separators, not decoded text.
  Repeat a pattern if a link interruption or board reset cuts it short.
*/

// Libraries provide the Arduino functions and the extra serial port.
#include <Arduino.h>
#include <AltSoftSerial.h>

// LCD support
// Set LCD_ENABLED to 0 for a temporary test using USB messages and LEDs.
#ifndef LCD_ENABLED
#define LCD_ENABLED 1
#endif
#if LCD_ENABLED
#include <LiquidCrystal.h>
LiquidCrystal lcd(A0, A1, A2, A3, A4, A5);
#endif

// Pin assignments and timing settings
// Keep both boards on the same link speed. Times below are milliseconds.
const uint8_t KEY_PIN = 2;
const uint8_t TX_LED_PIN = 4;
const uint8_t RX_LED_PIN = 5;
const uint8_t BUZZER_PIN = 6;
const uint8_t LINK_RX_PIN = 8;
const bool ENABLE_BUZZER = true;  // Set false for a quiet exercise.
const uint16_t BUZZER_HZ = 800;  // Tone pitch; try a new value after the lab.
const uint32_t USB_BAUD = 115200;
const uint32_t LINK_BAUD = 9600;
const uint32_t DEBOUNCE_MS = 15;
const uint32_t REFRESH_MS = 50;  // Repeat the current state every 0.05 s.
const uint32_t RX_TIMEOUT_MS = 500;  // Half a second with no valid input.

// Morse timing and display size
// Aim for 200 ms dots and 600 ms dashes. The cutoff is 400 ms.
// Keep these settings the same on both boards when practicing.
const uint32_t DOT_UNIT_MS = 200;
const uint32_t MIN_PRESS_MS = 50;  // Ignore very brief taps for the display.
const uint32_t DASH_THRESHOLD_MS = 2 * DOT_UNIT_MS;
const uint32_t LETTER_GAP_MS = 3 * DOT_UNIT_MS;
const uint32_t WORD_GAP_MS = 7 * DOT_UNIT_MS;
const uint8_t HISTORY_WIDTH = 13;  // 16 columns minus the three-character label.
const uint32_t LCD_STEP_MS = 5;
```

```
const uint8_t LCD_CHARS_PER_STEP = 2;

// Each direction keeps its own history and press timing.
// armed means a clean release has been seen; measuring means timing a hold.
struct MorseChannel {
  char history[HISTORY_WIDTH + 1];
  uint8_t length;
  bool armed;
  bool measuring;
  bool havePrevious;
  uint32_t pressAt;
  uint32_t releaseAt;
  uint32_t gapBeforePress;
  uint32_t symbols;
  uint32_t rejected;
};

// Declare these functions after the custom type so Arduino can compile them.
void resetMorse(MorseChannel &channel, bool armed);
void appendHistory(MorseChannel &channel, char value);
void observeMorse(MorseChannel &channel, bool down, uint32_t now, bool isTx);
void rememberLongIdle(MorseChannel &channel, bool down, uint32_t now);

// Create the peer port and separate outgoing and incoming Morse records.
AltSoftSerial peer;
MorseChannel txMorse = {};
MorseChannel rxMorse = {};

// Stored states and diagnostic counters
// rawDown is the latest sample; localDown is the debounced key state.
// remoteDown is the last valid state received from the other board.
bool rawDown = false;
bool localDown = false;
bool remoteDown = false;
bool incomingAlive = false;
bool everReceived = false;
bool soundOn = false;
uint32_t rawChangedAt = 0;
uint32_t lastTxAt = 0;
uint32_t lastValidRxAt = 0;
uint32_t sentBytes = 0;
uint32_t receivedBytes = 0;
uint32_t invalidBytes = 0;
#if LCD_ENABLED

// Remember the screen contents so unchanged characters need no rewrite.
char drawn[2][16];
uint8_t lcdScanIndex = 0;
uint32_t lastLcdAt = 0;
#endif

// Check a time interval without pausing the program.
// Unsigned subtraction also handles the millis() counter wrapping around.
bool elapsed(uint32_t now, uint32_t since, uint32_t interval) {
  return uint32_t(now - since) >= interval;
}

// Clear this channel's history and any incomplete press.
// The cumulative symbol and rejection counters are kept for diagnostics.
void resetMorse(MorseChannel &channel, bool armed) {
  channel.length = 0;
  channel.history[0] = '\0';
  channel.armed = armed;
  channel.measuring = false;
  channel.havePrevious = false;
  channel.pressAt = channel.releaseAt = channel.gapBeforePress = 0;
}
```

```
// Append one character; when full, slide the oldest character off the left.
void appendHistory(MorseChannel &channel, char value) {
  if (channel.length == HISTORY_WIDTH) {
    for (uint8_t i = 1; i < HISTORY_WIDTH; ++i)
      channel.history[i - 1] = channel.history[i];
    --channel.length;
  }
  channel.history[channel.length++] = value;
  channel.history[channel.length] = '\0';  // End the character string.
}

// Measure one complete press and classify it when the key is released.
// Repeated state refreshes do not restart the measurement.
void observeMorse(MorseChannel &channel, bool down, uint32_t now, bool isTx) {
  if (down) {
    // Start timing only a fresh press after a known release.
    if (channel.armed && !channel.measuring) {
      channel.measuring = true;
      channel.pressAt = now;
      // Cap a long gap at the word threshold; longer values are unnecessary.
      if (channel.havePrevious && channel.gapBeforePress < WORD_GAP_MS) {
        const uint32_t gap = uint32_t(now - channel.releaseAt);
        channel.gapBeforePress = gap < WORD_GAP_MS ? gap : WORD_GAP_MS;
      }
    }
    return;
  }
  channel.armed = true;  // A release establishes a clean starting point.
  if (!channel.measuring) return;  // No complete press to classify yet.
  channel.measuring = false;
  const uint32_t held = uint32_t(now - channel.pressAt);  // Measured hold duration.
  // A tiny tap may light an LED, but it does not become a Morse symbol.
  if (held < MIN_PRESS_MS) {
    ++channel.rejected;
    return;
  }
  // Add the preceding gap only when this new symbol is accepted.
  if (channel.havePrevious) {
    if (channel.gapBeforePress >= WORD_GAP_MS) {
      appendHistory(channel, ' ');
      appendHistory(channel, '/');  // A separator between words.
      appendHistory(channel, ' ');
    } else if (channel.gapBeforePress >= LETTER_GAP_MS) {
      appendHistory(channel, ' ');
    }
  }
  // Short hold = dot; long hold = dash. Both appear on release.
  const char symbol = held < DASH_THRESHOLD_MS ? '.' : '-';
  appendHistory(channel, symbol);
  channel.havePrevious = true;
  channel.releaseAt = now;
  channel.gapBeforePress = 0;
  ++channel.symbols;
  Serial.print(isTx ? F("TX SYMBOL ") : F("RX SYMBOL "));
  Serial.print(symbol);
  Serial.print(F(" hold_ms="));
  Serial.println(held);
}

// Remember a word-length pause even across a later millis() rollover.
void rememberLongIdle(MorseChannel &channel, bool down, uint32_t now) {
  if (!down && channel.havePrevious && !channel.measuring &&
      elapsed(now, channel.releaseAt, WORD_GAP_MS))
    channel.gapBeforePress = WORD_GAP_MS;
```

```
}

// Send one state byte and remember when it was sent.
// A refresh repeats the current state; it does not start another press.
void sendCurrentState(uint32_t now) {
  peer.write(localDown ? 'D' : 'U');
  lastTxAt = now;
  ++sentBytes;
}

// Read the button and reject brief contact bounce.
// Accept a new state only after the sampled input has settled for 15 ms.
void serviceButton(uint32_t now) {
  // Pressing connects the input to GND.
  const bool sampledDown = (digitalRead(KEY_PIN) == LOW);
  if (sampledDown != rawDown) {
    rawDown = sampledDown;
    rawChangedAt = now;  // Restart the settling timer after a change.
  }
  // A stable change becomes the new local state and is sent immediately.
  if (rawDown != localDown && elapsed(now, rawChangedAt, DEBOUNCE_MS)) {
    localDown = rawDown;
    sendCurrentState(now);
    observeMorse(txMorse, localDown, now, true);
    Serial.println(localDown ? F("LOCAL DOWN") : F("LOCAL UP"));
  }
  // Also handle a startup hold released before it could debounce.
  if (!rawDown && !localDown && elapsed(now, rawChangedAt, DEBOUNCE_MS))
    txMorse.armed = true;
}

// Clear a lost incoming link after 500 ms without a valid state byte.
// Discard an unfinished press instead of turning the timeout into a symbol.
void expireIncoming(uint32_t now) {
  // A missing update must not leave the remote output stuck on.
  if (incomingAlive && elapsed(now, lastValidRxAt, RX_TIMEOUT_MS)) {
    incomingAlive = false;
    remoteDown = false;
    resetMorse(rxMorse, false);  // Wait for a real release before timing a new press.
    Serial.println(F("RX LINK TIMEOUT - partial symbol discarded"));
  }
}

// Receive the other terminal's state.
// Process at most 16 bytes per pass so the key and outputs stay responsive.
void serviceReceiver(uint32_t now) {
  expireIncoming(now);
  for (uint8_t processed = 0; processed < 16 && peer.available(); ++processed) {
    const int value = peer.read();
    if (value < 0) break;  // No byte available.
    // Ignore anything outside our two-character protocol.
    if (value != 'D' && value != 'U') {
      ++invalidBytes;
      continue;
    }
    ++receivedBytes;
    lastValidRxAt = now;  // Only D/U bytes keep the incoming link alive.
    if (!incomingAlive) {
      incomingAlive = true;
      everReceived = true;
      remoteDown = false;
      // Wait for a real release before timing a new press.
      resetMorse(rxMorse, false);
      Serial.println(F("RX LINK UP"));
    }
    const bool newRemoteDown = value == 'D';
```

```
    if (newRemoteDown != remoteDown) {
      remoteDown = newRemoteDown;
      observeMorse(rxMorse, remoteDown, now, false);
      Serial.println(remoteDown ? F("REMOTE DOWN") : F("REMOTE UP"));
    } else if (!newRemoteDown) {
      rxMorse.armed = true;
    }
  }
}

// Update the LEDs and sound from the current states.
// The buzzer sounds if either key is held; overlapping holds share one tone.
void updateOutputs() {
  digitalWrite(TX_LED_PIN, localDown ? HIGH : LOW);
  digitalWrite(RX_LED_PIN, remoteDown ? HIGH : LOW);
  digitalWrite(LED_BUILTIN, incomingAlive ? HIGH : LOW);
  const bool wantSound = ENABLE_BUZZER && (localDown || remoteDown);
  // Start or stop the tone only when its required state changes.
  if (wantSound != soundOn) {
    soundOn = wantSound;
    if (soundOn) tone(BUZZER_PIN, BUZZER_HZ);
    else {
      noTone(BUZZER_PIN);
      digitalWrite(BUZZER_PIN, LOW);
    }
  }
}

// Refresh the display a little at a time.
// TX shows this key's history; RX shows the other key or a link message.
// Scan both rows fairly and write at most two changed characters per step.
void serviceLcd(uint32_t now) {
#if LCD_ENABLED
  if (!elapsed(now, lastLcdAt, LCD_STEP_MS)) return;
  lastLcdAt = now;
  const char *rows[2] = {txMorse.history, rxMorse.history};
  if (!incomingAlive) rows[1] = everReceived ? "LINK LOST" : "WAIT LINK";
  // A partial hold at startup or reconnection needs a release first.
  else if (!rxMorse.armed) rows[1] = "RELEASE KEY";
  uint8_t written = 0;
  for (uint8_t checked = 0; checked < 32 && written < LCD_CHARS_PER_STEP;
       ++checked) {
    const uint8_t row = lcdScanIndex / 16;
    const uint8_t col = lcdScanIndex % 16;
    lcdScanIndex = (lcdScanIndex + 1) % 32;  // Wrap after all 32 screen cells.
    char wanted;
    if (col == 0) wanted = row == 0 ? 'T' : 'R';
    else if (col == 1) wanted = 'X';
    else if (col == 2) wanted = ':';
    else {
      uint8_t length = 0;
      while (length < HISTORY_WIDTH && rows[row][length] != '\0') ++length;
      wanted = col - 3 < length ? rows[row][col - 3] : ' ';
    }
    // Write changed cells only to avoid clearing and flickering.
    if (drawn[row][col] != wanted) {
      lcd.setCursor(col, row);
      lcd.write(static_cast<uint8_t>(wanted));
      drawn[row][col] = wanted;  // Remember what was just written.
      ++written;
    }
  }
#else
  (void)now;
```

```
#endif
}

// Print a status snapshot to the computer, separate from the peer link.
// In Serial Monitor, send ? at 115200 baud to see these values.
void printStatus() {
  Serial.print(F("STATUS local="));
  Serial.print(localDown ? F("DOWN") : F("UP"));
  Serial.print(F(" remote="));
  Serial.print(remoteDown ? F("DOWN") : F("UP"));
  Serial.print(F(" rxLink="));
  Serial.println(incomingAlive ? F("UP") : F("NO RECENT DATA"));
  Serial.print(F("txBytes=")); Serial.print(sentBytes);
  Serial.print(F(" rxBytes=")); Serial.print(receivedBytes);
  Serial.print(F(" invalid=")); Serial.println(invalidBytes);
  Serial.print(F("TX:")); Serial.println(txMorse.history);
  Serial.print(F("RX:")); Serial.println(rxMorse.history);
  Serial.print(F("txSymbols=")); Serial.print(txMorse.symbols);
  Serial.print(F(" rxSymbols=")); Serial.println(rxMorse.symbols);
}

// One-time startup: configure pins, start communication, and set initial state.
void setup() {
  pinMode(KEY_PIN, INPUT_PULLUP);  // Released = HIGH, pressed = LOW.
  pinMode(TX_LED_PIN, OUTPUT);
  pinMode(RX_LED_PIN, OUTPUT);
  pinMode(BUZZER_PIN, OUTPUT);
  pinMode(LED_BUILTIN, OUTPUT);
  digitalWrite(TX_LED_PIN, LOW);
  digitalWrite(RX_LED_PIN, LOW);
  digitalWrite(BUZZER_PIN, LOW);
  digitalWrite(LED_BUILTIN, LOW);
#if LCD_ENABLED
  lcd.begin(16, 2);  // 16 character columns and two rows.
  for (uint8_t row = 0; row < 2; ++row)
    for (uint8_t col = 0; col < 16; ++col) drawn[row][col] = '\0';
#endif
  Serial.begin(USB_BAUD);  // Computer connection for diagnostics.
  peer.begin(LINK_BAUD);  // Separate connection to the other Uno.
  pinMode(LINK_RX_PIN, INPUT_PULLUP);  // Hold an unplugged RX input at idle HIGH.
  rawDown = digitalRead(KEY_PIN) == LOW;
  rawChangedAt = millis();
  resetMorse(txMorse, !rawDown);
  resetMorse(rxMorse, false);  // Wait for a real release before timing a new press.
  sendCurrentState(rawChangedAt);
  Serial.println(F("IEEE TELEGRAPH LCD"));
  Serial.println(F("USB 115200 | LINK 9600 | ? = status"));
}

// Repeat the small service tasks without a blocking delay().
// The 50 ms refresh maintains the link even when neither key changes.
void loop() {
  const uint32_t now = millis();
  serviceButton(now);
  serviceReceiver(now);
  if (elapsed(now, lastTxAt, REFRESH_MS)) sendCurrentState(now);
  updateOutputs();
  rememberLongIdle(txMorse, localDown, now);
  rememberLongIdle(rxMorse, remoteDown, now);
  serviceLcd(now);
  // USB console input is handled here and never forwarded to the peer.
  if (Serial.available() && Serial.read() == '?') printStatus();
}
```

How does everything work?

How the two Arduinos talk

Serial communication sends data one bit at a time. UART stands for Universal Asynchronous Receiver Transmitter. “Asynchronous” means the sender and receiver agree on a speed instead of sharing a separate clock wire. A start bit marks the beginning of each byte; a stop bit marks its end.

This link uses 9600 baud with 8 data bits, no parity bit and 1 stop bit, usually written 8N1. Here that means 9600 bits per second, including the framing bits. TX means transmit; RX means receive. A TX pin goes to the other board’s RX pin. The shared ground gives both boards the same voltage reference.

AltSoftSerial produces this UART-style link on Uno D9 and D8 using software and a timer. The ordinary Serial connection is separate and goes through USB to your computer at 115200 baud. Changing the Serial Monitor speed does not change the board-to-board speed.

What actually travels over the wire

Pressing the button sends the byte D, meaning down. Releasing sends U, meaning up. The program also repeats the current state every 50 ms. Repeated D bytes mean one continuing hold. After 500 ms without a valid update, the receiver clears the remote state so an interrupted link cannot leave a received hold on forever.

The transmitted bytes are D and U. Each terminal measures the time between state changes and turns completed holds into dots or dashes for its own display. A dot character is not what the serial link sends.

There is no delivery acknowledgment. A later refresh can repair the current held/released state, but it cannot recreate a whole tap lost while disconnected. Start the pattern again after a reset or interruption.

What a 1602A display does

“16 × 2” means sixteen character positions on each of two rows. The liquid crystal controls which character segments are visible; a separate LED backlight makes them easier to see. The potentiometer changes contrast at V0, so a lit display can still show no readable characters.

This display uses a parallel interface. In four-bit mode, four data wires carry half a byte at a time. RS selects an instruction or character data; E signals the display to accept the transfer. RW is grounded because this program only writes. LiquidCrystal handles the detailed timing.

A0–A5 can act as ordinary digital pins on an Uno, leaving D4, D5 and D6 available for the LEDs and piezo. LCD D0–D3 stay unused. This is a bare parallel LCD, so it has no I2C address; an I2C backpack is a different wiring arrangement.

How buttons and piezos work

INPUT_PULLUP keeps a released key HIGH. Pressing joins D2 to GND and makes it LOW. The 15 ms debounce interval filters contact chatter. An LED needs the correct polarity and its own series resistor. A passive piezo flexes as its voltage changes; tone(6, 800) produces an 800 Hz note. Use the supplied passive piezo rather than an unidentified speaker or active buzzer.

Either localDown or remoteDown can keep the piezo on. A timeout clears only the remote contribution. If your own key is still held, the sound correctly continues. Both directions can communicate at once, but taking turns makes the Morse easier to hear.

Morse Guide & Troubleshooting

The screen displays dots, dashes, and separators; it does not translate them into letters. Morse represents letters with short and long signals. A dot lasts one time unit; a dash lasts three. Leave one unit between signals in the same letter, three between letters and seven between words. These are total pauses, not pauses to add together.

Action	Standard length	Slow practice
Dot	1 unit	Hold about 0.2 s
Dash	3 units	Hold about 0.6 s
Pause inside a letter	1 unit	Release about 0.2 s
Pause between letters	3 units	About 0.8 s
Pause between words	7 units	About 1.6 s

For this lab, use roughly 0.8 s between letters and 1.6 s between words. That keeps your pauses comfortably above the software thresholds. A slash marks a word gap; it is never a replacement for a dot or dash. Once a row fills its 13 history positions, new characters push the oldest ones off the left.

Message	Written pattern	Read it as
HI	.... ..	H = four dots; I = two dots
SOS	... --- ...	S, O, S with letter pauses for practice

Spaces separate letters in this written notation. The traditional distress signal joins all nine marks as one continuous group (...---...).

When a terminal/LCD does not respond

Symptom	Check first
TX stays on with the button released	Button contact pair and breadboard rows; D2 may be shorted to GND.
One LED stays dark	LED polarity, its own resistor, the D4/D5 wire and ground return.
Local response works but remote does not	Both boards powered; D9 crosses to D8; common GND; 10 kΩ physical link resistors.
No sound or unexpected sound	Passive piezo type, polarity and 220 Ω path to D6; release both keys.
AltSoftSerial.h missing or upload fails	Install AltSoftSerial; select Uno and the correct port; use a USB data cable.

Symptom	Check first
Backlight on but text invisible	Slowly sweep contrast; check V0, VDD, VSS and the header connection.
Dark blocks or garbled characters	Check RS, E, RW to GND and the order of LCD D4–D7 to A2–A5.
Local symbols work but RX waits	Check D9 → 10 kΩ → other D8, common GND and peer power.
LED works but no dot or dash yet	Release the key; holds below 50 ms do not enter history.
Extra slashes appear	Your pauses reached 1.4 s. Keep shorter gaps within the same word.
The two screens disagree near a cutoff	Use clear 0.2 s dots and 0.6 s dashes; receiving and processing can shift measured timing slightly.